

이번 주 진행상황 - Stage1 이상구간 탐지

(1) 이번 주 진행상황 요약

기존 모델은 이상 채널(anomalous channel)을 직접 찾는 방식이었음.

이번엔 Stage1/Stage2로 역할을 나눠서 접근함.

- Stage1: 시계열에서 이상이 발생한 위치/구간(anomalous segment)을 탐지
- Stage2: Stage1이 찾은 구간을 바탕으로 어떤 채널에서 이상이 발생했는지 판별

Stage1의 구간이 정확해야 Stage2의 채널판별도 의미가 있음. 그래서 이번 주는 Stage1의 이상구간 탐지 성능 확인에 집중함.

비교를 위해 대표적인 다변량 이상탐지 SOTA 2개를 같이 정리함.

기존 SOTA의 이상 위치 / 이상 채널 처리 방식

TimeRCD (ICML 2026, zero-shot 파운데이션 모델)

이상 위치: 사전학습된 self-attention 기반 모델. test 시계열 자체를 문맥(context)으로 삼아 각 시점이 주변 문맥과 얼마나 다른지(relative context discrepancy)를 점수화함. train 데이터 없이 test만으로 동작(순수 zero-shot, entity별 학습 불필요).

이상 채널: 모델 내부적으로 채널별 로짓이 존재하나, 공개 API에서 채널 축을 평균내어 하나의 결합 점수만 반환함. 즉 채널판별 기능은 사실상 없음.

OmniAnomaly (KDD 2019, SMD 벤치마크 원 논문)

이상 위치: VAE+RNN으로 정상 패턴을 재구성(reconstruction)하고, 재구성확률이 낮은 시점을 이상으로 판단함. entity별 train 데이터로 학습 필요.

이상 채널: 채널별 재구성오차를 순위화(ranking)해서 HitRate@k, NDCG@k로 평가함. SMD의 interpretation_label(채널 GT)도 이 논문에서 처음 만듦. 채널판별을 부산물로 취급.

정리

두 SOTA 모두 '위치탐지'가 메인이고 '채널판별'은 부가기능이거나 아예 없음. 이번 주 Stage1 실험도 같은 위치탐지 과제를 놓고 비교함.

(2) Stage1 방법 - ① 설명 / ② 사용 이유 [1/2: DINOv2 계열]

1. DINOv2 patch-KNN (기존 방법) [DINOv2 사용 -- 원본 신호를 이미지로]

① 224픽(비중첩) 윈도우를 라인플롯 이미지로 그려 DINOv2에 입력, patch 임베딩을 train 윈도우들과 KNN 거리로 비교함. 38채널 점수 중 최댓값을 위치점수로 사용함.

② 이미지 기반 비전 모델이 시계열의 형태(shape) 이상을 잘 잡을 것으로 기대해서 시도함. 6개월간 구축한 핵심 백본이라 baseline으로 우선 확인함.

1-1. DINOv2 자체 변형 (같은 백본, 스코어링 방식만 다르게 시도) [전부 DINOv2 사용]

- self-referential testbank: train 없이 test 윈도우끼리만 서로 비교(GT/train 둘 다 불필요)

- patch-level 세분화: 윈도우 전체 합산 대신 16개 시간열(patch column) 단위로 세밀하게 채점

- 지속성 필터/최소지속길이 필터: 짧은 오탐을 후처리로 제거해서 정밀도 개선 시도

- 이 변형들은 baseline 대비 개선이 없거나(자체 참조, patch-level), held-out 검증에서 일반화에 실패함(지속성 필터) -> 자세한 수치는 결과표(3-1) 참고.

5. 탈주기 잔차 + DINOv2(mid-layer L8+L11) [DINOv2 사용 -- 탈주기 처리된 신호를 이미지로]

① 다음 페이지 3번에서 만든 탈주기 잔차 신호를 원본 대신 이미지로 그려서 DINOv2에 입력하되, 마지막 레이어 대신 8/11번째 레이어의 patch 임베딩 합을 사용해 KNN 점수를 계산함. 즉 '탈주기'는 DINOv2에 넣기 전 입력 신호를 바꾸는 전처리 역할만 하고, 점수 계산 자체는 DINOv2가 함.

② 탈주기+통계(다음 페이지 2,3,4번)만으로는 여전히 놓치는 사례(entity 내 이상 세기 편차)가 있어서, DINOv2의 형태 인식 능력을 탈주기 처리된 신호에 적용해 보완 가능한지 확인함.

(2) Stage1 방법 - ① 설명 / ② 사용 이유 [2/2: 순수 통계 계열]

2. z-score (통계 baseline) [DINOv2 사용 안 함 -- 순수 통계]

- ① 채널별 train 평균/표준편차 대비 test 값의 절대 z-score를 계산. 38채널 중 최댓값을 위치점으로 사용함. 이미지 렌더링도, DINOv2도 전혀 안 쓰고 원본 숫자만으로 계산함.
- ② DINOv2가 창(window) 단위로 뭉뚱그려서 짧은 스파이크형 이상을 놓치는 문제를 발견해서, 틱 단위로 정밀하게 반응하는 baseline이 필요해서 시도함.

3. 탈주기 잔차 (Phase-deseasonalization) [DINOv2 사용 안 함 -- 순수 통계]

- ① 채널별로 FFT로 주기를 추정하고, 주기 내 위상(phase)별 train 평균 프로파일을 test에서 빼서 잔차를 만듦. 이 잔차의 이동표준편차를 그대로 채널 점수로 씬(이미지나 모델 안 거침). 38채널 최댓값을 위치점으로 사용함. z-score와 마찬가지로 DINOv2를 전혀 안 씬.
- ② 특정 채널이 원래 습관적으로 주기적 스파이크를 내는 경우, 그 스파이크가 항상 최댓값으로 뽕혀 진짜 이상을 가리는 문제를 발견함. 습관적 패턴을 제거하고 남은 잔차만 보기 위해 시도함.

4. z-score + 탈주기 앙상블 [DINOv2 사용 안 함 -- 위 2번+3번, 순수 통계 둘끼리 결합]

- ① 위 2번(z-score)과 3번(탈주기 잔차) -- 둘 다 DINOv2 없이 만든 순수 통계 점수 -- 를 각각 rank 정규화한 뒤 최댓값을 취해 하나로 결합함. DINOv2는 이 조합 어디에도 안 들어감.
- ② 두 방법이 서로 다른 유형의 이상(전역적 크기 이상 vs 반복패턴 예외)을 잡아서, 단순 결합 만으로도 상호보완 효과를 기대함.

(3) 결과 - Stage1 이상구간 탐지 성능 (SMD 28개 entity, point-wise F1)

방법	DINOv2 사용?	평균 F1	비고
TimeRCD (SOTA)	X (별도 모델)	0.511	zero-shot 파운데이션 모델
DINOv2 patch-KNN	O (원본 신호)	0.376	기존 방법, 최하위
z-score	X	0.416	순수 통계
탈주기 잔차	X	0.527	순수 통계, 단독 최고
z-score + 탈주기 앙상블	X	0.535	순수 통계 둘끼리 결합, 전체 최고

탈주기+DINOv2(L8/L11)는 계산량 문제로 5개 entity만 확인함(다음 표 참고).

(3-1) 결과 - DINOv2 자체 변형 실험들

방법	DINOv2 사용?	평균 F1	확인 entity 수
DINOv2 patch-KNN (기본)	O (원본 신호)	0.376	28개
DINOv2 + 지속성필터(L=700)	O (원본 신호)	0.360	5개 (held-out 검증 실패)
DINOv2 self-ref testbank	O (원본 신호)	0.286	1개 (baseline보다 악화)
DINOv2 patch-level 세분화	O (원본 신호)	0.311	1개 (baseline보다 악화)

self-ref testbank/patch-level은 machine-1-1 1개 entity만 확인한 파일럿 수치임.
지속성필터(L=700)는 처음 6개 entity에선 F1=0.345까지 올랐으나, 진짜 held-out 5개로
재검증하니 0.360으로 오히려 baseline(0.399, 이 5개 기준)보다 낮아져 일반화 실패로 결론.

(3-2) 결과 - 탈주기 + DINOv2(mid-layer) 추가 효과 (5개 entity 기준)

방법	DINOv2 사용?	평균 F1	machine-3-2(주기채널 사례)
z-score + 탈주기	X	0.484	0.149
z-score + 탈주기 + DINOv2(L8/L14)	O (탈주기 처리된 잔차를 입력)	0.481	0.310 (2배 이상 개선)

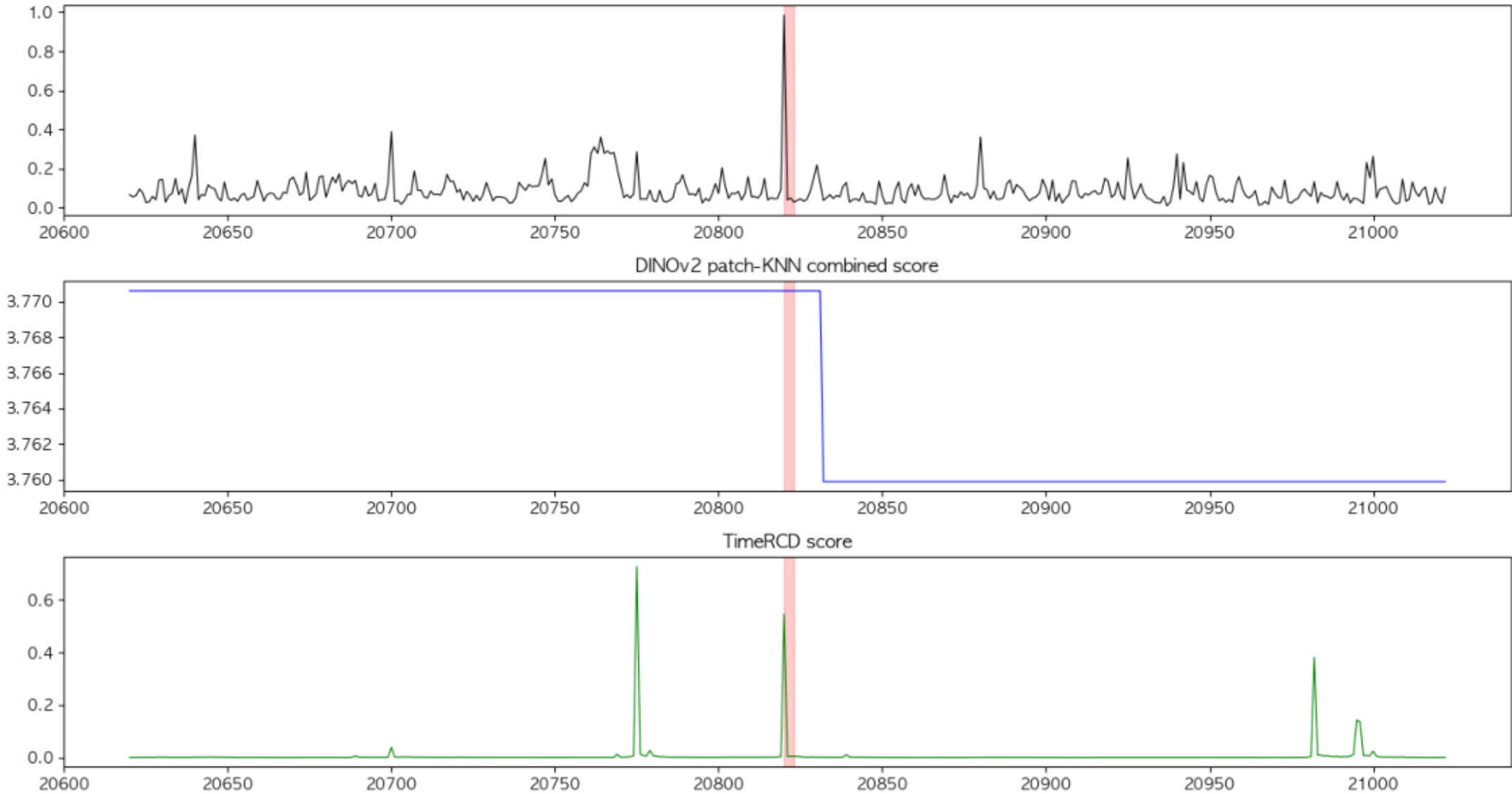
전체 평균은 근소하게 하락하지만, 목표했던 사례(주기적 스파이크 채널)는 크게 개선됨
 -> DINOv2가 특정 상황에는 유효하나, 단순 max 결합으로는 다른 entity에서의 손해가 더 큼.

(4) 원인분석 - DINOv2 실패 사례: 짧은 스파이크

machine-2-2 (20820,20823) 3틱 스파이크: TimeRCD recall=1.0 vs DINOv2 recall=0.0

DINOv2 대패 사례 (TimeRCD 1.0 vs DINOv2 0.0)

machine-2-2 ch11 (최고 z채널) raw 신호, GT=(20820,20823) len=3



DINOv2 실패 원인

어떤 상황에서 잘하지 못하는가

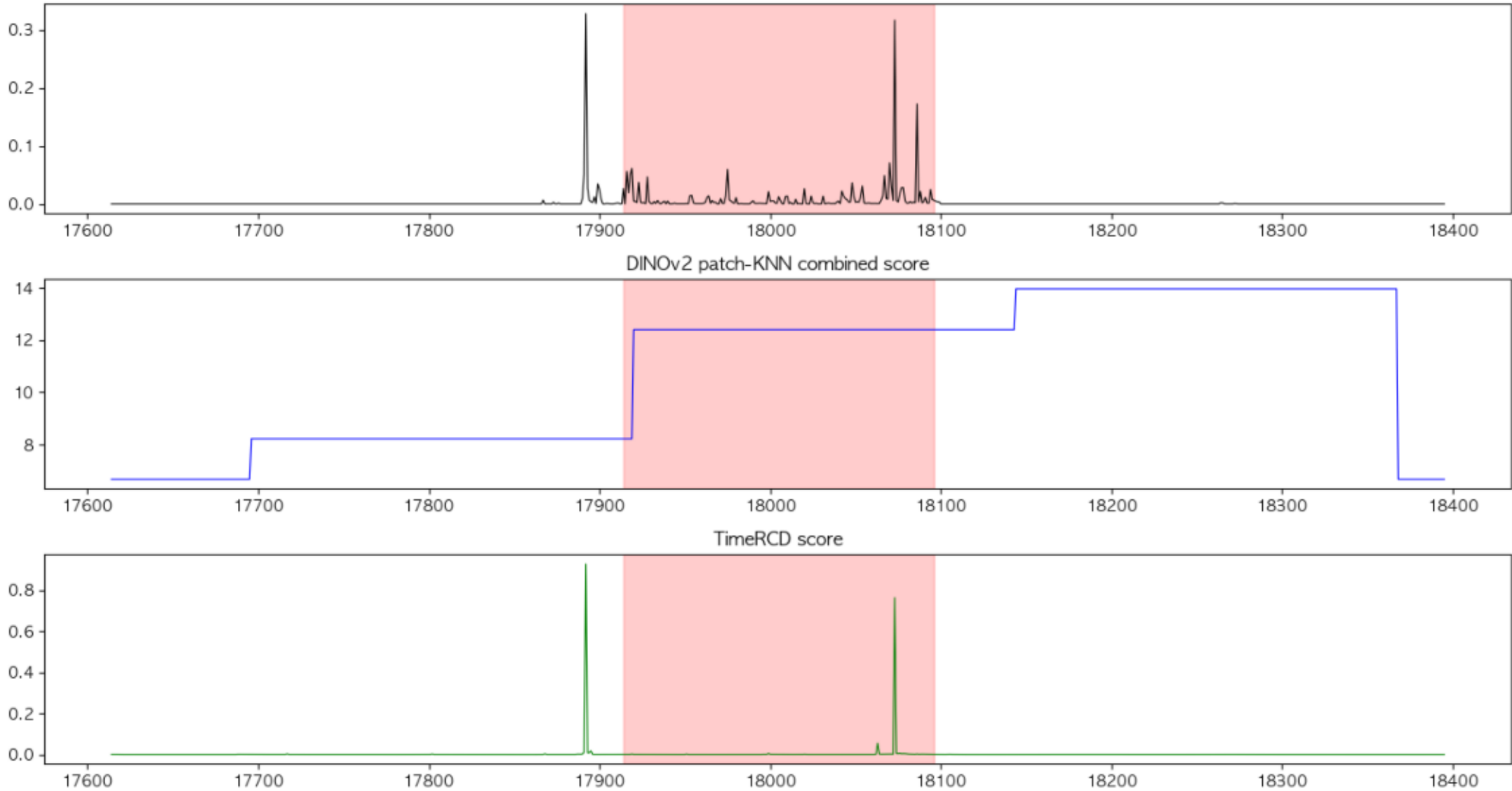
raw 신호엔 뚜렷한 스파이크가 있는데(맨 위 그래프), DINOv2 점수는 완전히 평평함(계단식).
224픽 창 하나를 통째로 집계하다 보니 3픽짜리 순간 스파이크가 창 전체 점수에 거의 영향을
못 줌. 즉 짧고 순간적인(spike형) 이상에서 recall이 크게 떨어짐. false negative가 이런
형태에서 반복적으로 발생함.

(4) 원인분석 - DINOv2 성공 사례: 밀도형 패턴

machine-2-9 [17914,18096) 182틱: DINOv2 recall=0.97 vs TimeRCD recall=0.01

DINOv2 대승 사례 (DINOv2 0.97 vs TimeRCD 0.01)

machine-2-9 ch9(최고 z채널) raw 신호, GT=[17914,18096) len=182



DINOv2 성공 원인

어떤 상황에서는 잘하는가

이 구간은 개별적으로는 크지 않은 자잘한 스파이크들이 구간 내내 이어지는 패턴임(밀도형).

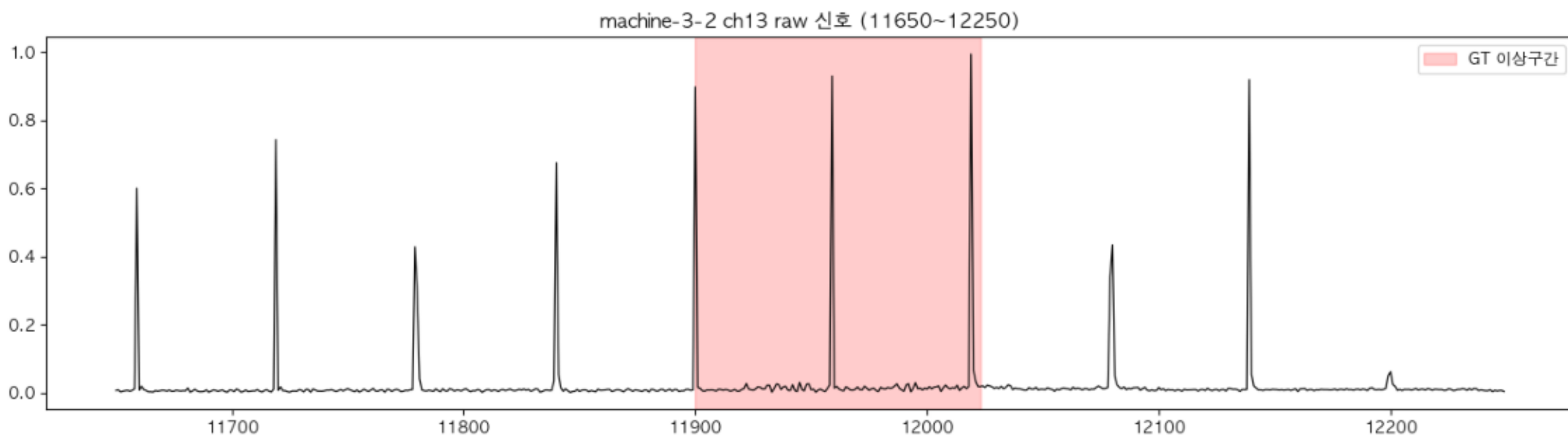
DINOv2는 창 전체를 누적해서 점수화하는 구조라 이런 '은은하게 계속되는 활동'을 잘 잡음.

TimeRCD는 국소적/순간적 변화에만 반응하는 구조라 이런 패턴을 거의 못 잡음.

-> DINOv2 feature는 순간적 크기보다 창 전체의 누적된 이질감(패턴 밀도 변화)을 잘 포착함.

(4) 원인분석 - 탈주기(최고성능 모델의 핵심)가 왜 필요했는가: 습관적 스파이크 채널

machine-3-2 ch13: GT 안팎으로 동일한 크기의 스파이크가 60틱마다 반복됨



z-score/DINOv2 공통 실패 원인 (탈주기 적용 전)

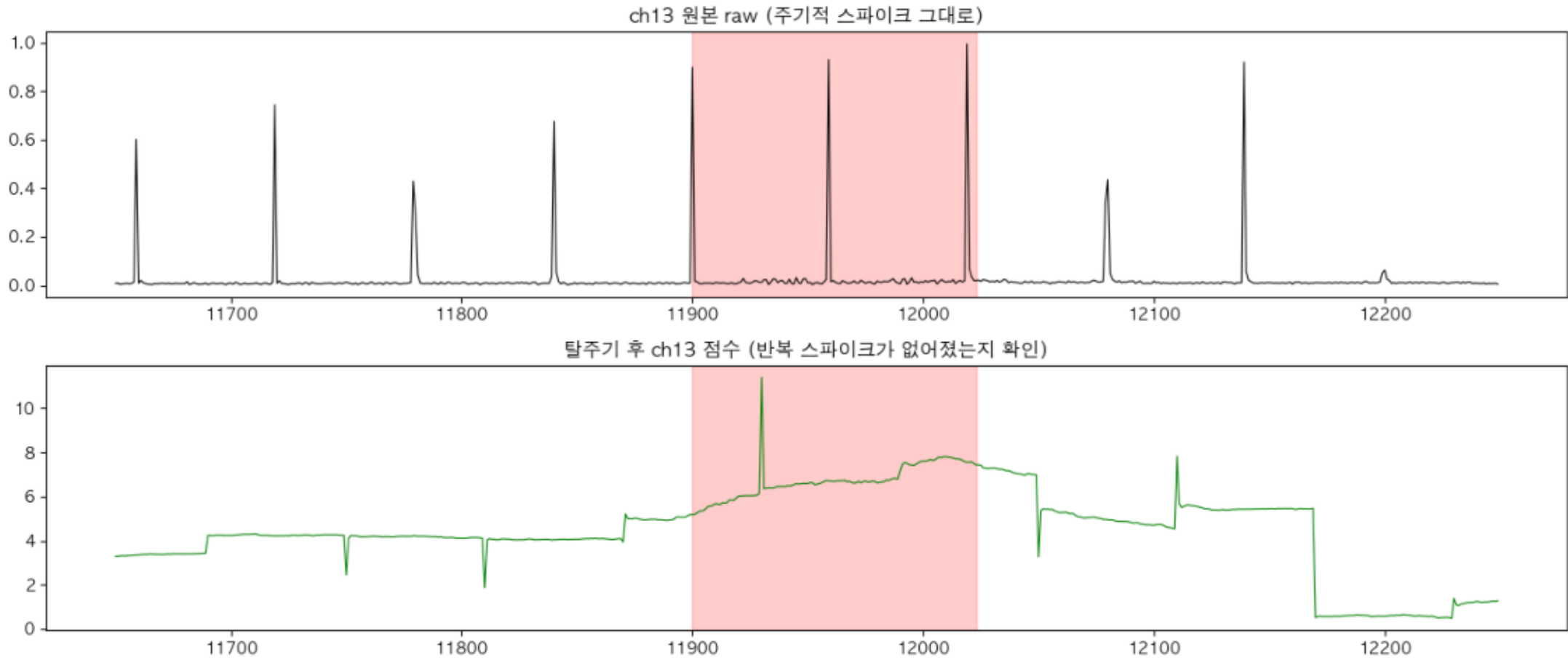
어떤 상황에서 잘하지 못하는가

ch13은 원래 60틱마다 습관적으로 크게 스파이크를 냄. 이상구간이든 아니든 똑같은 크기로 튀기 때문에, '38채널 중 가장 튀는 채널'을 고르는 max 방식에서 ch13이 항상 뽑힘.

그 결과 이상구간 밖에서는 false positive, 이상구간 안에서는 진짜 원인 채널(다른 채널의 미세한 신호)이 ch13에 가려져서 false negative가 같이 발생함.

(4) 원인분석 - 탈주기 적용 후: 문제 해결

같은 ch13, 탈주기 처리 후 - 반복 스파이크는 사라지고 이상구간 근처만 값이 올라감



탈주기 적용 후 개선

어떤 부분이 개선되는가

위상별 평균 프로파일(습관적 패턴)을 빼고 나니, 반복 스파이크는 잔차에서 거의 사라짐
(원본 그래프의 뾰족한 스파이크가 탈주기 후 그래프에선 안 보임).

대신 그 밑에 가려져 있던 신호 - 이상구간 전후로 잔차가 평소보다 흔들리는 구간 - 이 드러남.

이 방법을 z-score와 결합한 것이 이번 주 전체 최고 성능($F1=0.535$)을 기록함.

개선 방향

확인된 한계

1. z-score/탈주기 앙상블은 entity 안에 여러 이상이 섞여 있을 때(세기가 다른 이상들), entity 전체에 하나의 임계값만 쓰다 보니 약한 이상을 놓치는 경우가 있음.
2. 이 경우 DINOv2(특히 mid-layer 임베딩)가 여전히 유효한 신호를 갖고 있음을 확인함 (특정 entity에서 F1이 0.149 -> 0.310으로 2배 이상 개선).
3. 다만 단순 max 결합으로는 DINOv2의 노이즈가 다른 entity에서의 이득을 상쇄해버림.

다음 개선 방향

- 단순 max 앙상블 대신, 상황에 따라 방법별 가중치를 다르게 주는 결합 방식 검토 (예: entity/구간별로 어떤 방법이 신뢰할만한지 판단하는 선택적 결합).
- entity 내 이상 세기 편차 문제를 해결할 수 있는 적응적(entity-adaptive) 임계값 검토.
- Stage1 성능이 어느 정도 안정화되면, 이 결과를 Stage2(채널판별) 입력으로 연결.